Issuer freeze as a consensus rule

A proposal, and the analysis that changed its recommendation. Sequentia, 2026-08-12. For Andreas and Alberto.

Recommendation: do not build the rule as posed. Not because issuer control offends any principle of this network, and not because the change is hard to write. The analysis found six independent reasons it fails, and the first one is fatal on its own: **the rule does not deliver what was asked for.** A UTXO chain has no accounts to blacklist, and freezing a *script* is something a self-custody holder escapes by doing what their wallet already does automatically on every payment.

What a consensus freeze *can* do well is immobilise coins sitting at a **known, static address**: an exchange deposit address, a custodian, a bridge escrow. If that is the enforcement target, the rule is worth designing properly. If the target is an arbitrary self-custody holder, no UTXO-native design delivers it and the requirement needs renegotiating rather than implementing.

1. What was asked, and the constraint

The question was whether Sequentia should be able to do what an EVM blacklist upgrade does: bind holders who *already* hold an asset, at the issuer's instruction, without those holders having agreed to co-signed custody and without breaking Lightning for anyone else.

The owner's constraint on any such rule: freezability must be an **identifiable asset type**, fixed and visible at the asset's issuance, never a power every issuer silently holds over every asset. An asset not born freezable must never become freezable. That constraint is right, and it is assumed throughout what follows.

For completeness, what prompted it: the bridged USDC standard has two real gaps against Circle's FiatToken, a transfer-level pause and a transfer-level blacklist, and nothing on this chain closes them for a plain bearer asset. Co-signed custody (OpenAMP) closes them only for assets born into that model and is ruled out for USDC. So the question is whether consensus should.

2. Method

Six independent analyses were run against the actual trees, not against recollection: what the rule would break in the DEX, Lightning and atomic swaps; what it would break inside the node; whether either candidate design for "identifiable type" survives contact with the code; the abuse, denial-of-service and privacy surface; how other chains really implement issuer freeze; and an adversarial re-test of whether consensus is genuinely the only route. Findings rated blocker or serious were then attacked by separate reviewers instructed to refute them.

Two corrections to the record, since both bear on trust in the rest. First, an analyst reported that

Simplicity is inactive on every Sequentia chain. That is an artifact of reading the shared checkout `~/Sequentia`, which is stale on branch `pos-exprace`; master has it active, and it is live on testnet since height 89856, proven by a confirmed spend of a real Simplicity covenant in block 89860. **The shared checkout being stale is itself worth fixing**, because other sessions and agents read it. Second, an earlier draft of the bridged-USDC standard asserted a "bearer-asset model" principle as a reason not to pursue this. No such principle exists; that claim was mine and it was wrong.

3. The finding that decides it: a script is not an account

An EVM blacklist binds an **account**: a persistent identity that all of a holder's balance lives under, including everything they receive afterwards. The holder cannot rotate out of it. That property is what makes the mechanism worth having, and it is a property of the account model, not of blacklisting.

A UTXO chain has no accounts. The only thing a freeze can name is a **scriptPubKey**, and a scriptPubKey is rotated by definition on every spend: an ordinary HD wallet sends change to a fresh script automatically, without the user knowing. Worse, the freezing transaction is public in the mempool before it confirms, so the target sees it coming. The sequence is:

1. Issuer broadcasts a freeze naming script S.
2. Target, watching the mempool, spends from S to a fresh script S' in the same block or earlier.
3. The freeze confirms against an empty script. The coins are free, and the issuer must start again against a script it does not yet know.

So against a cooperative-but-sanctioned custodian who will not move funds, or a static deposit address, the rule works and works well. Against a self-custody holder who is actively evading, it is a race the holder wins by default. That is not a tuning problem; it is the difference between the two models.

Consequence for the ask. "Bind every existing holder, like an EVM blacklist" is not achievable on a UTXO chain by freezing scripts. It is achievable only by tracking taint forward through the transaction graph, which destroys fungibility and is a far larger change than the one proposed. The achievable semantics are: *immobilise the coins currently resting on a named script, and any coins later sent to it.* That sentence should be what any decision is taken against.

4. What it would break

4.1 Every multi-party transaction on this stack

Enforcement belongs in `Consensus::CheckTxInputs`, the one place both mempool acceptance and block validation pass through. That function has no per-input granularity: it walks every spent output and any rejection returns `state.Invalid()` for the **entire transaction**. Every value-carrying primitive this project has built deliberately packs several parties into one transaction:

Primitive	What one frozen input does to it

Lightning channel	The funding output is a 2-of-2 belonging to *neither* party alone. Freezing the frozen party's own script does nothing to an open channel; freezing the funding script strands the innocent peer completely, with no cooperative close, no force close and no HTLC resolution.
Same-chain DEX swap	One PSET carries both parties' inputs, so a freeze on either voids the whole swap, including the innocent side's inputs and outputs.
Covenant order book	A resting offer is one UTXO whose script is derivable from public order parameters. An issuer can compute and freeze it. The taker's fill is then rejected through no fault of their own, and the maker's timelocked refund dies with it.
Submarine swap	The LSP pays a Lightning invoice irreversibly, then claims an on-chain asset HTLC. A freeze landing between those two steps strands a third party who has already paid and has no recovery.

None of this is incidental. It is the direct consequence of freezing *scripts* in a system whose contracts deliberately hold value in scripts that belong to no single person.

4.2 Timelocks: a freeze that outlives a deadline is theft, not delay

This is the most serious finding after the semantics one. Lightning's safety rests on getting a transaction *confirmed* inside a bounded window: a revoked output must be swept within the peer's `to_self_delay`, and an HTLC must be timed out at its `cltv_expiry`. The covenant order book's refund and the cross-chain swap refunds are absolute `CLTV` branches.

An unbounded freeze over any of those scripts does not delay the remedy, it **destroys** it, and the loss lands on the counterparty rather than on the frozen party. Bounding freezes by a consensus-enforced maximum duration does not rescue this: the bound would have to be shorter than the shortest safety window of every protocol that will ever run on the chain, which is unknowable when the rule is written and short enough to make the freeze useless if guessed conservatively.

4.3 Confidentiality: the rule either leaks or is surrendered

A spent output's asset is a `CConfidentialAsset`. When the output is blinded it is a 33-byte commitment, and the asset id is simply not knowable to consensus. So a frozen holder's escape is trivial: hold the asset confidentially before the freeze lands, and enforcement can never see which asset the input carries.

Closing that hole means a consensus invariant that a freezable asset may **never** appear in a blinded output, enforced at issuance and at every output, forever. That is coherent and probably correct for a stablecoin, but it must be stated as what it is: a freezable asset permanently gives up opt-in confidentiality, and so, in any transaction that touches it, do its counterparties.

4.4 Node liveness: the rule as sketched would wedge block production

A spend that was valid when it entered the mempool becomes invalid the moment a freeze record confirms. Nothing evicts it: the mempool removes only transactions included in the block or directly conflicting with it, and a freeze record conflicts with nothing. The stale transaction is therefore re-selected into every block template, and template construction fails its final validity check, so producers skip slots. On a chain whose liveness depends on producers filling slots, that is a chain-wide outage triggered by one issuer's routine action.

Worse, and verified directly in this tree: `CTxMemPool::check()` re-runs `Consensus::CheckTxInputs` over every mempool entry inside a bare `assert`, carrying the comment "CheckTxInputs() should always pass" (`src/txmempool.cpp:920`). A rule whose verdict depends on state outside the transaction and its inputs breaks that documented invariant, and the node aborts.

Both are fixable, and neither is optional. Any freeze rule must ship with mempool eviction driven from block connection, re-admission on disconnect, and a reconsidered mempool sanity check. The sketch omitted all of it.

4.5 Neither candidate design for "identifiable type" survives as sketched

Option A, a derivation tag (freezable assets derive their id under a distinct constant, and a freeze record carries the entropy so consensus can re-derive). Cryptographically sound: forging it would be a second-preimage. But it fails twice. It carries *no authorisation whatsoever*, because the entropy is public on chain in the issuance transaction, so anyone could freeze anyone. And freezability becomes decidable only by someone holding the issuance transaction: the node has no asset-id-to-issuance index at all, so a holder with an asset id cannot answer "is this freezable?". Identifiable in theory, invisible in practice, which is exactly what the owner's constraint forbids.

Option B, a persisted registry of asset ids declared freezable at issuance. Its persistence cost is *lower* than assumed: the spent-pegin set is an exact precedent, living in the chainstate database and reverting on disconnect. Its real cost is different: the set is monotone and unprunable, since freezability must outlive the declaring output, and it is priced at one issuance input. That is permanent, unbounded state growth for a trivial fee.

If the rule proceeds, the workable shape is a hybrid neither option describes: commit the issuer's **freeze pubkey** into the asset id at issuance (giving type and authority in one move), and require a freeze record to be authorised by that key or by a reissuance-token spend in the same transaction.

4.6 Freezes inherit Bitcoin's reorg depth

Anchoring is supreme, so Sequentia reorganises whenever Bitcoin does. A freeze set derived from chain state must invert exactly on disconnect, and a deep enough anchor-driven reorg genuinely un-applies a freeze and re-enables a spend consensus had been rejecting. This is correct behaviour rather than a defect, but it means a freeze has no finality stronger than the parent chain, and an issuer must not be told otherwise.

5. What the alternatives actually deliver

Approach	Binds existing plain holders?	Cost
Covenant or Simplicity	No, and never can	Script validation has no chain-state oracle and an output's spend rule is fixed when it is created. Covenants bind only their own descendants.
Co-signed custody	Only coins already in enclaves	Every transfer needs the issuer online; ruled out for USDC by decision.
Relay policy only	No	The spend stays valid; any producer can include it in a block, and every node then accepts that block.
Bridge-side screening	No	Controls entry and exit, not circulation. Already implemented.
Consensus freeze	Coins at a named script, yes; a rotating holder, no	Everything in section 4.

So the tautology holds: consensus is the only layer that can touch an already-created plain output. The honest qualifier is that even at that layer, what it touches is a script rather than a person.

6. If it proceeds anyway: the minimum honest design

Stated so the cost is visible, not as a recommendation.

1. **Type and authority committed together at issuance.** The asset id commits to a freeze pubkey. No key, not freezable, ever. This fixes both the authorisation hole and the type constraint.
2. **Discoverability is part of the feature, not a follow-up.** An asset-id-keyed way to answer "is this freezable", and the wallet, explorer, registry and DEX all showing it before a user can acquire the asset. Without this the property is invisible and the constraint is not met.
3. **Freezable implies never blinded**, enforced inductively from issuance. Write down that such an asset has no confidentiality, for its holders or their counterparties.
4. **Mempool eviction on freeze, re-admission on unfreeze,** and a mempool sanity check that tolerates the new invalidity class. Non-negotiable; without it the chain stops making blocks.
5. **A stated policy for shared scripts.** Either refuse to freeze script types that cannot correspond to one holder, accepting the evasion that creates, or accept that freezes strand third parties in channels, swaps and resting orders. There is no third answer, and the choice belongs to the owner, not to the implementer.
6. **Height gate and coordinated cutover**, per CONTRIBUTING.md, because the rule rejects

spends and must not apply to history that predates it.

Scope estimate: a new consensus module, changes to issuance derivation and validation, mempool surgery, a startup rebuild path, RPCs, functional tests covering reorg and eviction, plus ecosystem display work across four repositories. This is a release of its own, not a patch, and it permanently enlarges the consensus surface, which is the scarcest thing the project has.

7. Options

1. **Do nothing.** USDC keeps the two documented deltas against FiatToken. The bridged standard already states them plainly, and Tether on Liquid operates this way today. **[Recommended for now.]**
2. **Scope the rule to static addresses and say so.** Design it honestly as "immobilise coins at a named script", target custodians, exchanges and escrows, and drop the claim that it binds arbitrary holders. Smaller, defensible, and still carries every section 4 cost except the semantic one.
3. **Build the full rule.** Only with sections 6.1 through 6.6 funded, and with the shared-script policy decided in advance.
4. **Test the requirement first.** Before any of the above, establish what an issuer actually requires. Every non-EVM chain Circle issues on has a freeze primitive, so it is plainly material to them, but nobody has yet asked them whether script-scoped freezing satisfies it. That answer determines whether option 2 is sufficient or option 3 is necessary. **[Cheapest next step, and it should come first.]**

8. Verification notes

Claims here were checked against the trees rather than recalled, and the two most load-bearing were re-checked by hand: the mempool assert at `src/txmempool.cpp:920`, and the whole-transaction rejection behaviour of `Consensus::CheckTxInputs`. The Lightning, DEX and swap findings cite the seqln, seqdex and covenant sources directly.

One analysis lens (issuer-freeze precedent on other chains) failed on a connection error and its conclusions are not relied upon here; the comparative claims retained are limited to ones sourced earlier and independently.

Prepared by Saba. Nothing in this proposal has been implemented. A working sketch of the rule was written during analysis and reverted unbuilt; no consensus code exists in any branch.